DJIRÉ Aboubakar Malick et ADJE Axel

Rapport-Projet-G1d06-IGI-3001

ROBOT EXPLORATEUR PROGRAMMABLE

TABLE DES MATIERES:

1. Description du projet.
2. Scénarios.
3. Explication du choix des configurations des GPIO et initialisations.
4. Code assembleur expliqué.
5. Difficultés rencontrées et solutions apportées.

Description du projet:

Notre robot nommé “Explora Bot-3000” est un drone terrestre concu pour faire de la reconnaissance dans des environnements risqués: champs de mines, zones contaminées, ou explorations extraterrestres. Le robot une fois mis sous tension entre dans un mode programmation ou l’utilisateur décidera du nombre n de pas qu’il souhaite que le robot fasse en cliquant sur le switch 1 n fois, il met ensuit fin à la phase de programmation en cliquant sur le switch 2.

Vient ensuite la phase d’execution où le robot équipé de capteurs avance tout en évitant les obstacles pour cartographier ou examiner une zone. La détection d'obstacles et la capacité à ajuster sa direction automatiquement sont des fonctionnalités programmées par les concepteurs (nous). Une fois que le robot a finit d’avancer le nombre de fois demandé, il s’arrête et entre de nouveau en mode programmation, prêt à recevoir de nouvelles consignes.

Scenarios:

Nous allons envisager 4 scénarios différents:

1. L'utilisateur n'appuis pas sur le switch 1 durant la phase de programmation

→ Dans ce cas le robot n'avance pas.

2. L'utilisateur appuis n fois sur le switch 1 durant la phase de programmation et il n'y a aucun obstacle sur la trajectoire du robot.

→ Dans ce cas le robot avance n fois.

3. L'utilisateur appuis n fois sur le switch 1 durant la phase de programmation et il y a un obstacle sur le coté gauche du robot.

→ Dans ce cas le robot avance jusqu'à l'obstacle, il le detecte grace au bumper gauche, allume uniquement la led droite et tourne à droite. Une fois qu'il a tourné, il ré-allume les deux LEDs, puis continu à avancer du nombre de pas qu'il lui reste à faire.

4. L'utilisateur appuis n fois sur le switch 1 durant la phase de programmation et il y a un obstacle sur le coté droit du robot.

→ Dans ce cas le robot avance jusqu'à l'obstacle, il le detecte grace au bumper droit, allume uniquement la led gauche et tourne à gauche. Une fois qu'il a tourné, il ré-allume les deux LEDs, puis continu à avancer du nombre de pas qu'il lui reste à faire.

Explication du choix des configurations des GPIO et initialisations:

code.s | config_led.s

```
;;; ADRESSES TROUVÉES DANS LA DATA SHEET.
; Adresse ou on choisit les ports pour lesquels on activera l'horloge
SYSCTL_PERIPH_GPIO EQU      0x400FE108

; configuration PWM
PWM_BASE        EQU     0x040028000
PWMENABLE       EQU     PWM_BASE+0x008

; Adresse de base du port F. Sert a configurer les data register.
GPIO_PORTF_BASE     EQU     0x40025000

; Adresse de base du port D. Sert a configurer les data register.
GPIO_PORTD_BASE     EQU     0x40007000

; Adresse de base du port E. Sert a configurer les data register.
GPIO_PORTE_BASE     EQU     0x40024000

; Adresse de base du port H. Va nous servir à controler le moteur gauche.
PORTH_BASE      EQU     0x40027000
GPIODATA_H      EQU     PORTH_BASE


; Configuration d'un pin en tant q'input ou output.
; Pour utiliser un pin comme entrée ou sortie, il faut configurer le bit correspondant dans l'adresse gpio input (ou output) direction du port.
GPIO_O_DIR          EQU     0x00000400

;Controle du courant.
GPIO_O_DR2R         EQU     0x00000500

; Digital enable register
; Pour utiliser un pin comme entrée ou sortie numérique, il faut configurer le bit correspondant dans l'adresse du gpio digital enable du port.
GPIO_O_DEN          EQU     0x0000051C

; Pul_up: configurer les boutons et switchs pour qu'ils soient détectés lorsque connecté à la terre.
GPIO_I_PUR          EQU     0x00000510

; Selection des pins
BROCHE4_5           EQU     0x30        ; led1 & led2 sur les pins 4 et 5 (0011 0000).

BROCHE6_7           EQU     0xC0        ; bouton poussoir 1 et bouton poussoir 2 sur les pins 6 et 7 (1100 0000).

BROCHE0_1           EQU     0x03 ;bumper 1 et bumper 2 sur les pins 0 et 1 (0000 0011).

GPIO_1              EQU     0x2 ;moteur sur le pin 1 (0000 0010)

;Durées
DUREE               EQU     0x002FFFFF ;durée affichage led

DUREE_D             EQU     0x9200c0 ; durée de virage
```

code.s | config_led.s

```
        AREA MyCode, CODE, READONLY
        ENTRY

        ;On exporte toute les fonctions qui seront utilisées dans le code principal.
        EXPORT LED_INIT
        EXPORT SWITCH_INIT
        EXPORT COUNT_CLICK
        EXPORT CLOCK_INIT
        EXPORT BUMPER_INIT
        EXPORT ReadState_BUMPER1
        EXPORT ReadState_BUMPER2


CLOCK_INIT
        ldr r6, = SYSCTL_PERIPH_GPIO
        mov r0, #0x00000038                     ;; Active l'horloge sur les GPIO D et F E où  sont branchés tous les périphériques utiles (0x38 == 0b111000)
        ; ;;                                                                                                                                (GPIO::FEDCBA)
        str r0, [r6]

        ; ;; "There must be a delay of 3 system clocks before any GPIO reg. access  (p413 datasheet de lm3s9B92.pdf)
        nop                                     ;; tres tres important....
        nop
        nop
        BX LR


LED_INIT
        ldr r6, = GPIO_PORTF_BASE+GPIO_O_DIR    ;; mettre à 1 les pins 4 et 5 (00110000) à l'adresse du gpio_output_dir du port F définissant ainsi les pins comme des outputs.
        ldr r0, = BROCHE4_5
        str r0, [r6]

        ldr r6, = GPIO_PORTF_BASE+GPIO_O_DEN    ;; mettre à 1 les pins 4 et 5 (00110000) à l'adresse du gpio_output_ditital_enable du port F définissant ainsi les pins comme des sorties numériques.
        ldr r0, = BROCHE4_5
        str r0, [r6]

        ldr r6, = GPIO_PORTF_BASE+GPIO_O_DR2R   ;; mettre à 1 les pins 4 et 5 (00110000) à l'adresse du gpio_output_dr2r du port F définissant ainsi l'intensité du courant de sortie à 2mA.
        ldr r0, = BROCHE4_5
        str r0, [r6]

        mov r2, #0x20
        mov r9, #0x10

        mov r3, #BROCHE4_5                      ;; va nous servir a allumer les deux LEDs
        ldr r6, = GPIO_PORTF_BASE + (BROCHE4_5<<2)  ;;définition du data register des LEDs (@data Register = @adresse_base + (mask<<2) ==> LED1&2)

        BX LR
```

```
SWITCH_INIT
        ldr r8, = GPIO_PORTD_BASE+GPIO_I_PUR    ;; Pul_up
        ldr r0, = BROCHE6_7
        str r0, [r8]

        ldr r8, = GPIO_PORTD_BASE+GPIO_O_DEN    ;; Enable Digital Function
        ldr r0, = BROCHE6_7
        str r0, [r8]

        ldr r8, = GPIO_PORTD_BASE + (BROCHE6_7<<2)  ;; @data Register = @base + (mask<<2) ==> SWITCH1&2

        BX LR


BUMPER_INIT
        ldr r7, = GPIO_PORTE_BASE+GPIO_I_PUR    ;; Pul_up
        ldr r0, = BROCHE0_1
        str r0, [r7]

        ldr r7, = GPIO_PORTE_BASE+GPIO_O_DEN    ;; Enable Digital Function
        ldr r0, = BROCHE0_1
        str r0, [r7]

        ldr r7, = GPIO_PORTE_BASE + (BROCHE0_1<<2)  ;; @data Register = @base + (mask<<2) ==> BUMPER1&2
```

Code assembleur principal:

code.s | config_led.s

```
        AREA MyCode, CODE, READONLY
        ENTRY
        EXPORT __main

        IMPORT  CLOCK_INIT                  ;initialise l'horloge sur tous les ports utilisés.
        IMPORT  LED_INIT                    ;initialise les LEDs.
        IMPORT  SWITCH_INIT                 ;initialise les SWITCHs.
        IMPORT  BUMPER_INIT                 ;initialise les BUMPERs.
        IMPORT  COUNT_CLICK                 ;fonction comptant le nombre de fois que le switch1 a été cliqué.

        IMPORT  MOTEUR_INIT                 ; initialise les moteurs (configure les pwms + GPIO)

        IMPORT  MOTEUR_DROIT_ON             ; activer le moteur droit
        IMPORT  MOTEUR_DROIT_OFF            ; déactiver le moteur droit
        IMPORT  MOTEUR_DROIT_AVANT          ; moteur droit tourne vers l'avant
        IMPORT  MOTEUR_DROIT_ARRIERE        ; moteur droit tourne vers l'arrière
        IMPORT  MOTEUR_DROIT_INVERSE        ; inverse le sens de rotation du moteur droit

        IMPORT  MOTEUR_GAUCHE_ON            ; activer le moteur gauche
        IMPORT  MOTEUR_GAUCHE_OFF           ; déactiver le moteur gauche
        IMPORT  MOTEUR_GAUCHE_AVANT         ; moteur gauche tourne vers l'avant
        IMPORT  MOTEUR_GAUCHE_ARRIERE       ; moteur gauche tourne vers l'arrière
        IMPORT  MOTEUR_GAUCHE_INVERSE       ; inverse le sens de rotation du moteur gauche

        IMPORT ReadState_BUMPER1
        IMPORT ReadState_BUMPER2

; r0 : réservé pour les programmes temporaires
; r1 : réservé au compteur de clicks
; r2 : pour allumer uniquement la LED2
; r3 : réservé a l'état allumer les deux LEDs
; r4 : réservé pour les programmes temporaires
; r6 : réservé au data register LEDs
; r7 : réservé au data register BUMPERS
; r8 : réservé au data register SWITCHES
; r9 : pour allumer uniquement la LED1
; r10: réservé pour les programmes temporaires
; r12: réservé pour les programmes temporaires
```

code.s | config_led.s

```
__main
        BL CLOCK_INIT
        BL LED_INIT
        BL SWITCH_INIT
        BL BUMPER_INIT

        BL MOTEUR_INIT


programmation ; Phase de programmation.
        BL COUNT_CLICK


execution     ; Phase d'exécution.
        SUBS r1, r1, #1
        BNE avancer ;tant que le compteur contenant le nombre de fois ou le switch1 a été cliqué n'est pas égale à 0, avancer.
        B programmation ;sinon retourner en mode programmation.


avancer
        STR r3, [r6]
        BL MOTEUR_DROIT_ON
        BL MOTEUR_GAUCHE_ON
        BL MOTEUR_DROIT_AVANT
        BL MOTEUR_GAUCHE_AVANT
        BL wait
        BL wait
        BL MOTEUR_GAUCHE_OFF
        BL MOTEUR_DROIT_OFF
        BL wait
        BL wait

        B execution


wait
        LDR R12, =0x86470
waitloop
        PUSH {LR}
        BL ReadState_BUMPER1 ; lire l'état du bumper1.
        BL ReadState_BUMPER2 ; lire l'état du bumper2.
        POP {LR}
        SUBS R12, R12, #1
        BNE waitloop
        BX LR

        nop
        END
```

code.s | config_led.s

```

COUNT_CLICK
        ;Compte le nombre de fois ou le switch 1 est pressé durant la phase de programmation.
        mov r1, #0x00 ; initialisation du compteur de click.
        mov r12, #0x00
        str r12, [r6] ; on éteint les deux LEDs


countloop
        LDR r10,[r8] ; lecture du data register des switchs
        CMP r10,#0x80 ; verification que le switch 1 a été pressé, si c'est le cas alors le pin 6 passe a 0, on a alors (1000 0000) dans le data register des switchs.
        BEQ click ; branchement vers la procédure à suivre en cas de click

        CMP r10,#0x40 ; verification que le switch 2 a été pressé
        BNE countloop ; loop tant que le switch 2 n'a pas été pressé
        add r1, r1, #1
        BX LR


click
        add r1, r1, #1 ; on incrémente le compteur de click de 1.
        str r9, [r6]   ; on allume la LED1.

        ldr r4, = DUREE                        ;; durée pendant laquelle la LED1 va rester allumée
wait1   subs r4, r4, #1; mini boucle d'attente
        bne wait1
        mov r12, #0x00; on éteint les LEDs
        str r12, [r6]


        b countloop
```

code.s | config_led.s

```
ReadState_BUMPER1
        ;Vérifie si le bumper1 à été touché.
        ldr r10,[r7] ; bumper 1 appuyé
        CMP r10,#0x02
        BNE RETURN   ; retour vers le programme principal si le bumper n'est pas appuyé.
        STR r2, [r6] ; allumage de la LED2
        B   TOURNER_GAUCHE


ReadState_BUMPER2
        ;Vérifie si le bumper2 à été touché.
        ldr r10,[r7] ; bumper 2 appuyé
        CMP r10,#0x01
        BNE RETURN   ; retour vers le programme principal si le bumper n'est pas appuyé.
        STR r9, [r6] ; allumage de la LED1
        B   TOURNER_DROITE

RETURN
        BX LR


TOURNER_GAUCHE
        ;Fait tourner le moteur gauche en arrière.
        ldr r11, =(GPIODATA_H+(GPIO_1<<2))
        mov r0, #2
        str r0,[r11]

        LDR r0,= DUREE_D
wait2   subs r0, #1; mini boucle d'attente.
        bne wait2

        ;Fait tourner le moteur gauche de nouveau en avant.
        ldr r11, =(GPIODATA_H+(GPIO_1<<2))
        mov r0, #0
        str r0,[r11]
        BX LR


TOURNER_DROITE
        ;Fait tourner le moteur droite en arrière.
        ldr r11, =(GPIO_PORTD_BASE+(GPIO_1<<2))
        mov r0, #0
        str r0,[r11]

        LDR r0,= DUREE_D
wait3   subs r0, #1; mini boucle d'attente.
        bne wait3

        ;Fait tourner le moteur gauche de nouveau en avant.
        ldr r11, =(GPIO_PORTD_BASE+(GPIO_1<<2))
        mov r0, #2
        str r0,[r11]
        BX LR
```

```
        nop
        END
```

Difficultés rencontrées et solutions apportées:

→ Le moteur recule durant la phase de programmation.

Les moteurs reculent automatiquement lorsqu'ils sont allumés, nous avons donc allumés les moteurs uniquement lorsqu'on etait sur le point de les utiliser.

→ Dans le fichier code lors de la boucle wait, on verifie si les bumpers sont appuyés, mais une fois que c'est fait, il n'arrive plus a continuer l'execution normale du programme. Lors de la vérification du statut des bumpers, on a fait un branchement: "BL ReadState_BUMPER1", et si le bumper est éffectivement appuyé, on a fait un branchement utilisant "BL", ce qui a override le registre "LR". Le code ne peut donc pas retourner au point de départ.

2 Solutions:

Soit ne plus utiliser "BL" pour éviter les conflits et réécrire entièrement la procédure pour faire tourner le robot.

Soit utiliser PUSH {LR} et POP {LR} pour enregistrer le contenu du registre "LR" et pouvoir le ré-utiliser plus tard.

→ La première phase de programmation ne fonctionne pas, c'est à dire que juste après avoir mis le robot sous tension, lorsqu'on appuis sur le switch1 n fois et qu'on valide en appuyant sur le switch2, le robot n'avance pas.

Nous n'avons pas réussi à régler ce problème, il faut ainsi sauter la première phase de programmation, en appuyant directement sur le switch2 après la mise sous tension.

→ On a remarqué que lorsqu'on vérifiait l'état des périphériques a l'intérieur des boucles d'attente, comme dans wait (ligne 76), le temps d'exécution augmente, les boucles d'attente prennent beaucoup plus de temps à se terminer.

Pour remédier à cela nous avons juste diminué la valeur dans le registre à décrémenter: r1 dans le cas de wait.

→ Au début de la conception du code, nous avons souvent rencontré des bugs innatendus, cela était du a des conflits entre les registres. Nous utilisions un registre pour faire quelque chose, mais il était modifié à notre ainsu quelque part d'autre, ce qui a conduit à énormément de bugs.

Pour éviter ce genre de problème, nous avions deux choix, configurer les registres à chaque fois qu'on voulait les utiliser.

OU

Tenir compte de quel registre est utilisé pour quoi. C'est ce que nous avons fait à la ligne 28.

→ Nous avons aussi rencontré un bug majeur cette fois ci avec la boucle d'attente wait, elle ne se terminait jamais.

Il se trouvait que nous avions changé l’indicateur z lors de la verification des switchs et des bumpers. Pour remédier à ce problème, nous avons remarqué que l’instruction subs doit être mise après les les branchements pour éviter que l’indicateur z se fasse override durant l’exécution du code de vérification des périphériques.